



Univerzitet u Novom Sadu
FAKULTET TEHNIČKIH NAUKA

Departman za industrijsko inženjerstvo i inženjerski menadžment

Inženjerstvo informacionih sistema



**Sistem za vođenje evidencije zaposlenih, godišnjih odmora, odsustva, radnih i putnih
naloga, praksi i dodatne edukacije**

Projektna dokumentacija iz predmeta

Upravljanje razvojem informacionih sistema

Profesor: Vladimir Mandić

Asistent: Andrej Katin

Studenti: Jovana Zečević IT9/2022

Ksenija Mudri IT15/2022

Anastasija Berić IT38/2022

Nikolina Đukić IT42/2022

Nataša Tanasković IT54/2022

Sara Vujičić IT80/2022

Novi Sad, april 2026.

Sadržaj

Opis problema i ciljeva sistema	5
Ciljevi sistema	6
Očekivane koristi sistema	7
Korišćene tehnologije	8
.NET	8
C#	9
PostgreSQL	10
React	10
Jira	11
GitHub	11
Docker	12
Microsoft Azure for Students.....	12
Opis korisničkih uloga.....	13
Funkcionalni i nefunkcionalni zahtevi.....	16
Funkcionalni zahtevi	16
Nefunkcionalni zahtevi	19
Arhitektura sistema	21
Opšti pregled arhitekture	21
Mikroservisna struktura	21
Komunikacija između servisa.....	24
Slojna organizacija servisa.....	25
Upravljanje podacima	25
Bezbednost i kontrola pristupa.....	26
Prednosti izabrane arhitekture	26
Modelovanje domena kroz mikroservisnu arhitekturu	27
Opis mikroservisa.....	32
Servisi	33
Dokumentovanje API-ja	35
Opis baze podataka	43

Struktura podataka	43
Integritet i konzistentnost podataka.....	44
Izolacija podataka po servisima	45
Pristup bazi podataka.....	46
Bezbednost podataka	46
Opis komunikacije između servisa.....	46
Opis frontend-backend interakcije.....	47
Opis DevOps postavke.....	49
Opis testiranja.....	50
Zaključak	54

Sadržaj slika

Slika 1. logo .NET platforme.....	8
Slika 2. logo C# programskog jezika	9
Slika 3. Prikaz razvojnog okruženja - Visual Studio.....	9
Slika 4. logo PostgreSQL	10
Slika 5. logo React-a	11
Slika 6. Logo Jira	11
Slika 7. logo GitHub-a	12
Slika 8. logo Docker-a	12
Slika 9. logo Microsoft Azure.....	13
Slika 10. Model 1: Entiteti i veze	28
Slika 11. Model 2: Agregati i agregatni koreni <Root>	30
Slika 12. Model 3: Kompletan model mikroservisne arhitekture.....	32
Slika 13. Dokumentacija API-ja za entitet LeaveBalance	36
Slika 14. Dokumentacija API-ja za entitet Department.....	37
Slika 15. Dokumentacija API-ja za entitet Notification	37
Slika 16. Dokumentacija API-ja za entitet Employee	38
Slika 17. Dokumentacija API-ja za entitet Admin	38
Slika 18. Dokumentacija API-ja za entitet Manager.....	39

Slika 19. Dokumentacija API-ja za entitet Request	39
Slika 20. Dokumentacija API-ja za entitet TravelOrder.....	40
Slika 21. Dokumentacija API-ja za entitet Internship.....	40
Slika 22. Dokumentacija API-ja za entitet Education.....	41
Slika 23. Dokumentacija API-ja za entitet Absence.....	41
Slika 24. Dokumentacija API-ja za entitet AnnualLeave	42
Slika 25. Dokumentacija API-ja za entitet SickLeave.....	42
Slika 26. Dokumentacija API-ja za entitet DayOff	43
Slika 27. Baza RequestDb u pgAdmin.....	44
Slika 28. Prikaz kolona baze RequestDb	45
Slika 29. Prikaz GitHub repozitorijuma	49
Slika 30. Klasa AnnualLeaveTests, kreiranje entiteta i validacije.....	50
Slika 31. Prikaz pojedinih xUnit testova klase AnnualLeaveTests.....	51
Slika 32. Prikaz pojedinih testova unutar Absence.Domain.Tests projekta	51
Slika 33. Prikaz integracionog testa klase AbsencesControllerTests za upload fajlova	52

Opis problema i ciljeva sistema

U savremenim organizacijama upravljanje podacima o zaposlenima, njihovim odsustvima, godišnjim odmorima, putnim nalogima, praksama i edukacijama često je fragmentisano i neefikasno. U mnogim slučajevima, ovi podaci se vode kroz različite nepovezane sisteme ili čak manuelno (npr. Excel tabele, e-mail komunikacija), što dovodi do brojnih problema.

Jedan od ključnih problema je nepostojanje centralizovanog sistema za evidenciju zaposlenih i njihovih aktivnosti. Ovo otežava pristup informacijama, povećava mogućnost grešaka i dovodi do neusklađenosti podataka. Na primer, evidencija godišnjih odmora može biti neprecizna, što može rezultirati preklapanjem odsustava ili pogrešnim obračunom preostalih dana.

Proces odobravanja zahteva (odsustva, putni nalozi, edukacije i sl.) često nije standardizovan. Zahtevi se šalju putem e-maila ili usmeno, što otežava praćenje njihovog statusa i istorije. Takođe, nedostatak transparentnosti može dovesti do nesporazuma između zaposlenih, menadžera i administracije.

Dodatni problem predstavlja nedostatak uvida u sve aktivnosti zaposlenog na jednom mestu. Bez jedinstvenog pregleda (npr. kroz kalendar), zaposleni i menadžeri nemaju jasnu sliku o planiranim odsustvima, poslovnim putovanjima ili edukacijama.

Bez adekvatnog sistema notifikacija, korisnici često nisu blagovremeno obavešteni o promenama statusa zahteva, što dodatno usporava procese i smanjuje efikasnost organizacije.

Sve navedeno ukazuje na potrebu za jedinstvenim informacionim sistemom koji će omogućiti centralizovano, pouzdano i transparentno upravljanje svim relevantnim podacima i procesima vezanim za zaposlene.

Ciljevi sistema

Osnovni cilj sistema je razvoj centralizovanog informacionog rešenja koje omogućava efikasno upravljanje zaposlenima i svim njihovim poslovnim aktivnostima.

Specifični ciljevi sistema su:

- Omogućiti centralizovanu evidenciju zaposlenih i njihove pripadnosti organizacionim jedinicama.
- Digitalizovati proces podnošenja i obrade zahteva (odsustva, putni nalozi, edukacije, prakse).
- Uvesti standardizovan i transparentan proces odobravanja zahteva kroz uloge zaposlenog, menadžera i administratora.
- Omogućiti precizno praćenje godišnjih odmora, uključujući raspoložive i prenete dane.
- Sprečiti greške kao što su preklapanje odsustava ili neispravni podaci kroz validacije sistema.
- Obezbediti pregled svih aktivnosti zaposlenog kroz jedinstveni kalendar.
- Omogućiti evidenciju i praćenje edukacija i stručnih praksi zaposlenih.
- Implementirati sistem notifikacija kako bi korisnici bili pravovremeno obavešteni o važnim događajima.
- Obezbediti siguran pristup sistemu putem autentifikacije i autorizacije korisnika.

Očekivane koristi sistema

Implementacijom ovakvog sistema ostvaruju se brojne koristi za organizaciju:

- **Povećana efikasnost** – automatizacija procesa smanjuje potrebu za manuelnim radom i ubrzava obradu zahteva.
- **Smanjenje grešaka** – validacije i centralizovani podaci smanjuju rizik od netačnih informacija.
- **Veća transparentnost** – svi zahtevi i njihove promene statusa su jasno vidljivi i dokumentovani.
- **Bolja organizacija resursa** – menadžeri imaju uvid u raspoloživost zaposlenih i mogu bolje planirati aktivnosti.
- **Jednostavan pristup informacijama** – svi podaci se nalaze na jednom mestu i lako su dostupni.
- **Poboljšana komunikacija** – sistem notifikacija omogućava pravovremeno informisanje korisnika.
- **Podrška donošenju odluka** – dostupnost tačnih i ažurnih podataka omogućava kvalitetnije upravljanje.
- **Skalabilnost sistema** – sistem se može proširivati novim funkcionalnostima u skladu sa potrebama organizacije.

Korišćene tehnologije

U razvoju sistema za vođenje evidencije zaposlenih, godišnjih odmora, odsustva, radnih i putnih naloga, praksi i dodatne edukacije korišćen je savremeni skup tehnologija koji omogućava izgradnju pouzdanog, skalabilnog i efikasnog informacionog sistema zasnovanog na mikroservisnoj arhitekturi. Izbor tehnologija usmeren je ka stabilnim i široko prihvaćenim rešenjima koja obezbeđuju visoke performanse, sigurnost i jednostavno održavanje sistema.

Backend deo sistema implementiran je korišćenjem .NET platforme i programskog jezika C#, dok je za skladištenje podataka korišćena relacionala baza PostgreSQL. Za razvoj korisničkog interfejsa upotrebljena je React biblioteka, koja omogućava izradu modernih i interaktivnih web aplikacija. Pored toga, korišćeni su alati za upravljanje projektom, verzionisanje koda i kontejnerizaciju aplikacije, čime je obezbeđena bolja organizacija razvoja i lakša distribucija sistema.

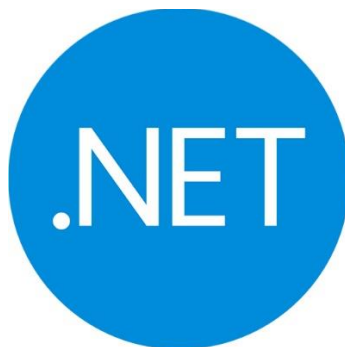
Kombinacija ovih tehnologija omogućava jasnu podelu sistema na nezavisne komponente, koje se mogu razvijati, testirati i skalirati odvojeno, što je posebno važno u mikroservisnoj arhitekturi.

.NET

.NET predstavlja modernu razvojnu platformu namenjenu izgradnji različitih tipova aplikacija, uključujući web servise i distribuirane sisteme. U okviru ovog projekta korišćen je za implementaciju backend mikroservisa koji obezbeđuju REST API interfejsse i upravljaju poslovnom logikom sistema.

Platforma omogućava efikasno rukovanje HTTP zahtevima, obradu podataka i integraciju sa bazama podataka, što je čini pogodnom za razvoj složenih informacionih sistema. Njene ključne prednosti su visoke performanse, sigurnost i bogat ekosistem alata.

U projektu je korišćena verzija .NET 8, koja donosi unapređenja u performansama i podršku za razvoj cloud i mikroservisnih aplikacija.

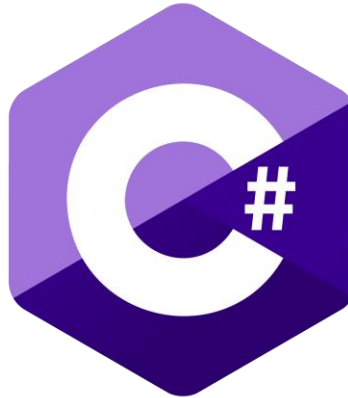


Slika 1. logo .NET platforme

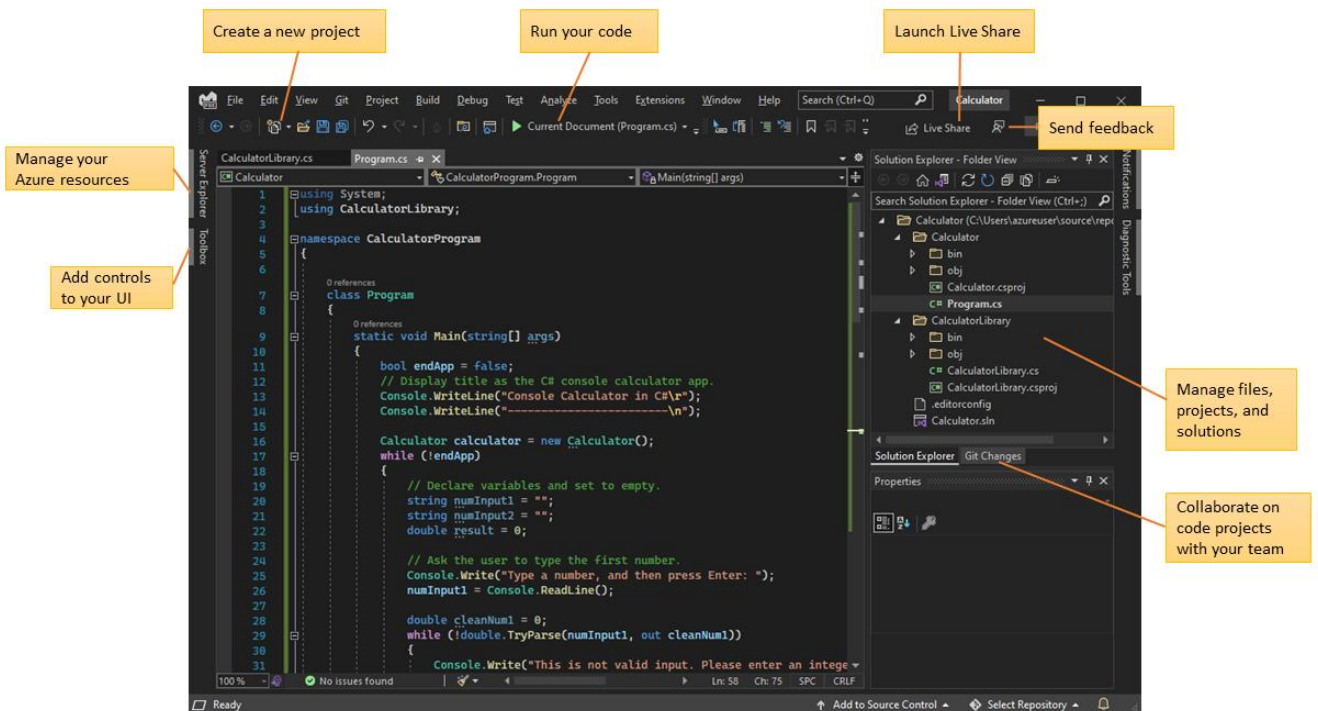
C#

C# je objektno-orijentisani programski jezik koji se koristi u okviru .NET platforme. U ovom projektu korišćen je za implementaciju poslovne logike, definisanje domen modela (npr. Employee, Request, Absence) i realizaciju komunikacije između slojeva sistema.

Jezik omogućava jaku tipizaciju, jednostavno održavanje koda i podršku za savremene programske paradigme, što doprinosi razvoju pouzdanih i skalabilnih aplikacija.



Slika 2. logo C# programskog jezika



Slika 3. Prikaz razvojnog okruženja - Visual Studio

PostgreSQL

PostgreSQL je open-source relacioni sistem za upravljanje bazama podataka koji se odlikuje visokom pouzdanošću i podrškom za kompleksne operacije nad podacima. U projektu se koristi za trajno skladištenje svih podataka vezanih za zaposlene, zahteve, odsustva, edukacije i druge entitete sistema.

Ova baza omogućava očuvanje integriteta podataka kroz relacije i ograničenja, kao i rad sa velikim količinama podataka. Njena stabilnost i skalabilnost čine je pogodnom za poslovne sisteme.



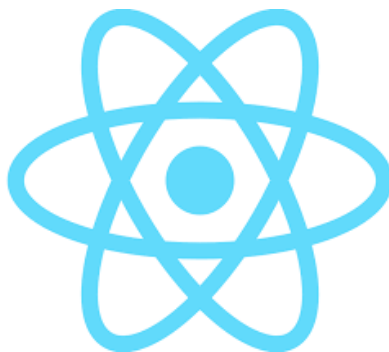
Slika 4. logo PostgreSQL

React

Frontend deo sistema razvijen je korišćenjem React biblioteke, koja omogućava izgradnju dinamičnih i interaktivnih korisničkih interfejsa. React se zasniva na komponentnoj arhitekturi, što omogućava ponovno korišćenje koda i lakše održavanje aplikacije.

U okviru projekta implementirana je Single Page Application (SPA), gde se komunikacija sa backend servisima vrši putem API poziva, dok se prikaz podataka dinamički ažurira bez ponovnog učitavanja stranice.

Ovakav pristup omogućava bolje korisničko iskustvo, brži odziv sistema i efikasnije upravljanje prikazom podataka, kao što su kalendar aktivnosti zaposlenog i lista zahteva.



Slika 5. logo React-a

Jira

Jira je alat za upravljanje projektima i praćenje zadataka koji je razvila kompanija Atlassian. U okviru ovog projekta korišćen je za planiranje, organizaciju i praćenje razvoja sistema, kao i za koordinaciju rada članova tima. Jira omogućava strukturisano evidentiranje zadataka, definisanje prioriteta i praćenje napretka kroz različite faze razvoja. U datom [LINK](#) možete videti kako je to izgledala podela zadataka našeg tima i Jira Dashboard tima.

Alat podržava agilne metodologije razvoja, kao što su Scrum i Kanban, čime omogućava transparentno upravljanje sprintovima, backlogom i statusima zadataka. Korišćenjem Jire obezbeđena je bolja komunikacija unutar tima, jasna raspodela odgovornosti i kontinuiran uvid u tok razvoja projekta, što doprinosi efikasnijem i organizovanijem radu na izgradnji sistema.



Slika 6. Logo Jira

GitHub

GitHub je platforma za verzionisanje koda zasnovana na Git sistemu. U projektu je korišćen za čuvanje izvornog koda, praćenje izmena i timsku saradnju.

Korišćenjem grana omogućeno je paralelno razvijanje različitih funkcionalnosti, dok su pull request-ovi korišćeni za pregled i kontrolu kvaliteta koda pre njegovog spajanja u glavnu granu.



Slika 7. logo GitHub-a

Docker

Docker je platforma za kontejnerizaciju aplikacija koja omogućava pokretanje sistema u izolovanom okruženju zajedno sa svim zavisnostima. U projektu je korišćen za pakovanje mikroservisa i baze podataka u kontejnere.

Ovakav pristup omogućava konzistentnost između razvojnog i produkcionog okruženja, kao i jednostavno pokretanje sistema na različitim mašinama.



Slika 8. logo Docker-a

Microsoft Azure for Students

Microsoft Azure for Students predstavlja cloud platformu koja omogućava hostovanje aplikacija i korišćenje cloud resursa u obrazovne svrhe. U ovom projektu korišćen je za postavljanje backend servisa i omogućavanje pristupa sistemu putem interneta.

Azure pruža skalabilnu i pouzdanu infrastrukturu, kao i alate za upravljanje aplikacijama, što omogućava stabilan rad sistema u realnim uslovima.



Slika 9. logo Microsoft Azure

Opis korisničkih uloga

Informacioni sistem za vođenje evidencije zaposlenih, godišnjih odmora, odsustva, radnih i putnih naloga, praksi i dodatne edukacije predviđa više tipova korisnika sa različitim pravima pristupa i odgovornostima. Sistem implementira model upravljanja pristupom zasnovan na ulogama (RBAC – Role-Based Access Control), čime se obezbeđuje da svaki korisnik ima pristup samo onim funkcionalnostima koje su u skladu sa njegovom ulogom u organizaciji.

Na osnovu analize poslovnih procesa i definisanih korisničkih zahteva, identifikovane su sledeće osnovne korisničke uloge:

Zaposleni (Employee)

Zaposleni predstavlja osnovnog korisnika sistema koji koristi funkcionalnosti vezane za lične zahteve i pregled sopstvenih podataka.

Zaposleni ima mogućnost:

- prijave na sistem (login)
- pregleda ličnih podataka i statusa naloga
- podnošenja zahteva za odsustvo
- podnošenja zahteva za godišnji odmor
- evidentiranja bolovanja
- podnošenja zahteva za slobodan dan
- podnošenja zahteva za putni nalog

- prijave za dodatne edukacije
- uvida u stanje godišnjeg odmora (leave balance)
- pregleda ličnog kalendara aktivnosti
- praćenja statusa podnetih zahteva
- dobijanja notifikacija o promenama statusa zahteva

Sistem omogućava zaposlenom pristup isključivo sopstvenim podacima i zahtevima, čime se obezbeđuje zaštita privatnosti i integritet informacija.

Menadžer (Manager)

Menadžer je korisnik koji ima ulogu odobravanja i kontrole zahteva zaposlenih u okviru organizacije.

Menadžer može:

- pregledati listu zahteva zaposlenih
- odobriti ili odbiti zahtev (odsustvo, putni nalog, edukacija i dr.)
- promeniti status zahteva
- proslediti zahtev administratoru na dalje procesiranje (u zavisnosti od tipa zahteva)
- primati notifikacije o novim zahtevima koji zahtevaju odobrenje
- pratiti aktivnosti zaposlenih u okviru svog domena odgovornosti

Sistem obezbeđuje da menadžer može da upravlja samo zahtevima koji pripadaju zaposlenima iz njegovog organizacionog domena.

Administrator (Admin)

Administrator predstavlja korisnika sa najvišim nivoom pristupa u sistemu i odgovoran je za celokupno upravljanje sistemom i podacima.

Administrator može:

- kreirati, ažurirati i brisati korisničke naloge zaposlenih
- upravljati podacima o zaposlenima i odeljenjima
- pregledati i uređivati sve zahteve u sistemu
- finalno odobravati ili odbijati zahteve (ukoliko je deo procesa)
- upravljati evidencijom godišnjih odmora i leave balance-a

- pregledati istoriju aktivnosti zaposlenih
- upravljati praksama i edukacijama
- imati uvid u sve entitete sistema
- obezbediti administrativnu kontrolu i audit promena

Administrator ima pun pristup sistemu, ali sistem i dalje zadržava logiku razdvajanja odgovornosti kroz različite nivoe odobravanja.

Sistem (System / Automated Processes)

Pored korisničkih uloga, sistem uključuje i automatske procese koji izvršavaju poslovna pravila bez direktne interakcije korisnika.

Sistem automatski:

- proverava validnost unosa (npr. obavezna polja, datumi)
- sprečava preklapanje odsustava
- ažurira status zahteva nakon odobrenja ili odbijanja
- ažurira stanje godišnjeg odmora nakon odobravanja
- briše prenete dane nakon isteka roka (npr. 30. jun)
- generiše notifikacije o promenama statusa
- ažurira kalendar zaposlenog na osnovu aktivnosti

Ova uloga obezbeđuje konzistentnost podataka i automatizaciju poslovnih pravila.

Korisnik (User – apstraktna uloga)

Osnovna apstraktna uloga predstavlja svakog registrovanog korisnika sistema. Nakon autentifikacije, korisniku se dodeljuje konkretna uloga (zaposleni, menadžer ili administrator), na osnovu koje se definišu prava pristupa.

Sistem podržava:

- autentifikaciju korisnika
- autorizaciju na osnovu uloge
- upravljanje sesijom
- kontrolu pristupa funkcionalnostima sistema

Funkcionalni i nefunkcionalni zahtevi

Na osnovu analize poslovnih procesa sistema za vođenje evidencije zaposlenih, godišnjih odmora, odsustva, radnih i putnih naloga, praksi i dodatne edukacije, identifikovani su funkcionalni i nefunkcionalni zahtevi koji definišu ponašanje sistema i kriterijume njegovog kvaliteta.

Funkcionalni zahtevi

Funkcionalni zahtevi opisuju konkretne funkcionalnosti koje sistem mora da obezbedi kako bi podržao rad organizacije i potrebe svih korisničkih uloga.

1. Upravljanje korisnicima i zaposlenima

Sistem mora omogućiti:

- prijavu i autentifikaciju korisnika (login/logout)
- razlikovanje korisničkih uloga (zaposleni, menadžer, administrator)
- upravljanje podacima o zaposlenima
- kreiranje, izmenu i deaktivaciju korisničkih naloga (admin funkcija)
- dodelu zaposlenih odeljenjima
- jedinstvenu identifikaciju svakog zaposlenog

2. Upravljanje zahtevima (Request sistem)

Sistem mora omogućiti:

- podnošenje zahteva od strane zaposlenog
- definisanje tipa zahteva (odsustvo, godišnji odmor, putni nalog, edukacija, praksa)
- praćenje statusa zahteva (pending, approved, rejected)
- pregled istorije zahteva
- vezivanje zahteva za konkretnog zaposlenog
- automatsko ažuriranje statusa zahteva
- evidenciju ko je obradio zahtev i kada

3. Upravljanje odsustvima i odmorima

Sistem mora omogućiti:

- podnošenje zahteva za različite tipove odsustva
- evidenciju godišnjeg odmora
- sprečavanje preklapanja datuma odsustva
- unos početnog i krajnjeg datuma odsustva
- automatsko ažuriranje stanja godišnjeg odmora nakon odobrenja
- evidenciju bolovanja i slobodnih dana

4. Upravljanje putnim nalogima

Sistem mora omogućiti:

- podnošenje zahteva za putni nalog
- unos destinacije i vremenskog perioda putovanja
- vezivanje putnog naloga za zaposlenog
- obavezno odobravanje pre realizacije
- čuvanje istorije putnih naloga

5. Upravljanje edukacijama i praksama

Sistem mora omogućiti:

- prijavu zaposlenih za dodatne edukacije
- evidenciju trajanja i naziva edukacije
- odobravanje edukacija
- povezivanje edukacija sa zaposlenim
- evidenciju praksi (internship-a)
- povezivanje praksi sa mentorima

6. Leave Balance modul

Sistem mora omogućiti:

- prikaz godišnjeg odmora za izabranu godinu
- prikaz ukupnog broja dana odmora
- prikaz prenetih dana iz prethodne godine
- prikaz preostalih dana
- automatsko brisanje prenetih dana nakon isteka roka (npr. 30. jun)

7. Kalendarski pregled aktivnosti

Sistem mora omogućiti:

- prikaz ličnog kalendara zaposlenog
- prikaz svih aktivnosti (odsustva, putovanja, edukacije, prakse)
- automatsko ažuriranje kalendara nakon promene statusa
- jasno označavanje tipova aktivnosti

8. Notifikacije

Sistem mora omogućiti:

- slanje notifikacija zaposlenima
- obaveštavanje o promeni statusa zahteva
- prikaz razloga i tipa odluke
- praćenje svih važnih događaja u sistemu

9. Administracija sistema

Administrator mora imati mogućnost:

- upravljanja svim korisnicima i zaposlenima
- pregleda i upravljanja svim zahtevima
- upravljanja odeljenjima i organizacionom strukturom
- uvida u sve podatke sistema
- kontrole i audit-a promena u sistemu

Nefunkcionalni zahtevi

Nefunkcionalni zahtevi definišu kvalitativne karakteristike sistema, odnosno kako sistem treba da funkcioniše.

1. Performanse

- Sistem mora omogućiti brz odziv pri radu sa podacima
- Obrada zahteva mora se izvršavati u razumnom vremenskom okviru
- Sistem mora podržati veći broj istovremenih korisnika
- Kašnjenje u prikazu podataka mora biti minimalno

2. Bezbednost

- Sistem mora obezbediti autentifikaciju korisnika
- Pristup funkcionalnostima mora biti kontrolisan na osnovu uloge (RBAC)
- Osetljivi podaci moraju biti zaštićeni od neovlašćenog pristupa
- Lozinke moraju biti čuvane u hash-ovanom obliku
- Sistem mora obezbediti sigurnu komunikaciju između klijenta i servera

3. Pouzdanost

- Sistem mora obezbediti konzistentnost podataka
- Mora sprečiti gubitak podataka u slučaju grešaka
- Mora obezbediti pravilno rukovanje izuzecima
- Sistem mora biti stabilan tokom kontinuiranog rada

4. Upotrebljivost

- Korisnički interfejs mora biti intuitivan i jednostavan za korišćenje
- Funkcionalnosti moraju biti logično organizovane
- Sistem mora omogućiti lako snalaženje bez posebne obuke
- Prikaz podataka mora biti jasan i pregledan

5. Skalabilnost

- Sistem mora omogućiti dodavanje novih modula bez većih izmena
- Mora podržati rast broja korisnika i podataka
- Arhitektura mora biti pogodna za mikroservisno proširenje

6. Održivost i proširivost

- Kod sistema mora biti modularan i lako održiv
- Sistem mora omogućiti integraciju sa drugim servisima
- Poslovna pravila moraju biti lako promenljiva bez velikih izmena sistema

Arhitektura sistema

Sistem za upravljanje ljudskim resursima projektovan je kao distribuirani informacijski sistem zasnovan na mikroservisnoj arhitekturi. Ovakav pristup omogućava jasno razdvajanje poslovnih domena, jednostavnije održavanje sistema i nezavisno razvijanje pojedinačnih funkcionalnosti.

Opšti pregled arhitekture

Arhitektura sistema sastoji se od više međusobno povezanih mikroservisa, pri čemu je svaki servis odgovoran za određenu poslovnu oblast. Komunikacija između servisa realizovana je putem REST API interfejsa korišćenjem HTTP protokola i JSON formata za razmenu podataka.

Prednosti ovakvog pristupa su:

- modularnost sistema
- jednostavnije održavanje i razvoj
- mogućnost nezavisnog skaliranja servisa
- izolacija grešaka
- lakša integracija novih funkcionalnosti
- bolja organizacija poslovne logike

Mikroservisna struktura

Sistem je podeljen na više domen-specifičnih mikroservisa, pri čemu svaki servis poseduje sopstvenu poslovnu logiku, podatke i API.

1. EmployeeService

EmployeeService predstavlja centralni servis za upravljanje zaposlenima.

Njegove funkcionalnosti uključuju:

- evidenciju zaposlenih
- kreiranje, izmenu i brisanje podataka o zaposlenima
- upravljanje korisničkim nalogima
- autentifikaciju i prijavu korisnika
- upravljanje korisničkim ulogama (Employee, Manager, Admin)
- povezivanje zaposlenih sa zahtevima, odsustvima i drugim evidencijama

Ovaj servis predstavlja osnovu sistema jer su svi ostali podaci povezani sa zaposlenima.

2. DepartmentService

DepartmentService upravlja organizacionom strukturom kompanije.

Njegove funkcionalnosti obuhvataju:

- evidenciju odeljenja
- dodeljivanje zaposlenih odeljenjima
- promenu pripadnosti zaposlenog odeljenju
- pregled zaposlenih po odeljenjima
- čuvanje istorije promena organizacione strukture

Servis obezbeđuje da svaki zaposleni bude povezan sa tačno jednim odeljenjem.

3. RequestService

RequestService upravlja svim zahtevima koje zaposleni mogu podneti kroz sistem.

Podržani tipovi zahteva uključuju:

- odsustvo
- godišnji odmor
- bolovanje
- slobodan dan
- putni nalog
- edukaciju
- praksu

Funkcionalnosti servisa uključuju:

- kreiranje zahteva
- praćenje statusa zahteva
- odobravanje i odbijanje zahteva
- evidenciju datuma obrade zahteva
- evidenciju osobe koja je obradila zahtev
- čuvanje istorije promena statusa

Status zahteva može imati vrednosti:

- Pending
- Approved
- Rejected

RequestService predstavlja centralnu komponentu za upravljanje poslovnim procesima odobravanja.

4. AbsenceService

AbsenceService upravlja evidencijom svih vrsta odsustava zaposlenih.

Njegove funkcionalnosti uključuju:

- evidenciju godišnjih odmora
- evidenciju bolovanja
- evidenciju slobodnih dana
- proveru perioda odsustva
- sprečavanje preklapanja datuma odsustava
- povezivanje odsustava sa zaposlenima
- dostavljanje podataka za prikaz u kalendaru zaposlenog

Servis osigurava konzistentnost podataka vezanih za odsustva i njihovu usklađenost sa poslovnim pravilima.

5. LeaveBalanceService

LeaveBalanceService upravlja stanjem godišnjeg odmora zaposlenih.

Njegove funkcionalnosti uključuju:

- vođenje evidencije godišnjeg odmora po godinama
- prikaz ukupnog broja dana odmora
- evidenciju prenetih dana iz prethodne godine
- automatsko umanjenje raspoloživog broja dana nakon odobrenja odmora
- izračunavanje preostalog broja dana
- automatsko uklanjanje neiskorišćenih prenetih dana nakon definisanog roka

Ovaj servis implementira poslovna pravila vezana za korišćenje godišnjeg odmora.

6. NotificationService

NotificationService zadužen je za obaveštavanje korisnika o važnim događajima u sistemu.

Njegove funkcionalnosti uključuju:

- slanje obaveštenja zaposlenima
- obaveštavanje o promeni statusa zahteva
- obaveštavanje o odobrenim i odbijenim zahtevima
- čuvanje istorije notifikacija
- dostavljanje informacija koje zahtevaju pažnju korisnika

Servis omogućava pravovremeno informisanje korisnika o svim relevantnim promenama u sistemu.

Komunikacija između servisa

Mikroservisi međusobno komuniciraju korišćenjem REST API interfejsa i HTTP/HTTPS protokola.

Razmena podataka realizuje se u JSON formatu.

Karakteristike komunikacije:

- servis-to-servis komunikacija
- jasno definisani API endpoint-i
- nezavisnost servisa
- mogućnost horizontalnog skaliranja
- tolerancija na greške

Na primer:

- EmployeeService komunicira sa DepartmentService radi povezivanja zaposlenih sa odeljenjima.
- RequestService koristi podatke iz EmployeeService kako bi povezao zahteve sa zaposlenima.
- AbsenceService komunicira sa RequestService tokom obrade zahteva za odsustvo.
- LeaveBalanceService ažurira stanje godišnjeg odmora nakon odobrenja zahteva u RequestService.

- NotificationService prima informacije od RequestService i šalje odgovarajuća obaveštenja korisnicima.
- EmployeeService koristi NotificationService za slanje sistemskih obaveštenja korisnicima.

Slojna organizacija servisa

Svaki mikroservis implementira internu slojevitú arhitekturu.

API sloj (Presentation Layer)

- REST kontroleri
- obrada HTTP zahteva i odgovora
- validacija ulaznih podataka
- autentifikacija i autorizacija

Poslovni sloj (Business Layer)

- implementacija poslovnih pravila
- obrada zahteva
- koordinacija između servisa

Sloj pristupa podacima (Data Access Layer)

- komunikacija sa bazom podataka
- repozitorijumi
- ORM mapiranje

Model i DTO sloj

- domen modeli
- DTO objekti za razmenu podataka

Upravljanje podacima

Svaki mikroservis poseduje sopstvenu bazu podataka.

Ovakav pristup omogućava:

- autonomiju servisa
- veću sigurnost podataka

- jednostavnije izmene šeme baze
- nezavisno skaliranje baza podataka
- smanjenje međusobnih zavisnosti

Bezbednost i kontrola pristupa

Bezbednost sistema zasniva se na autentifikaciji korisnika i kontroli pristupa zasnovanoj na ulogama (RBAC).

Definisane su sledeće uloge:

- Employee
- Manager
- Admin

Prava pristupa:

- Employee može da kreira i pregleda sopstvene zahteve
- Manager može da odobrava zahteve zaposlenih iz svog tima
- Admin ima pristup svim podacima i funkcionalnostima sistema

Za autentifikaciju se koristi JWT mehanizam zasnovan na tokenima.

Prednosti izabrane arhitekture

Predložena mikroservisna arhitektura omogućava:

- jasnu podelu poslovnih domena
- jednostavnije održavanje sistema
- nezavisno razvijanje funkcionalnosti
- lakše testiranje i deployment
- visoku skalabilnost
- bolju bezbednost i kontrolu pristupa
- mogućnost budućeg proširenja sistema

Arhitektura je u potpunosti usklađena sa definisanim korisničkim pričama i omogućava implementaciju svih funkcionalnih zahteva projekta za upravljanje ljudskim resursima i administracijom zaposlenih.

Modelovanje domena kroz mikroservisnu arhitekturu

U cilju pravilnog projektovanja mikroservisne arhitekture, primenjen je pristup zasnovan na principima **Domain-Driven Design (DDD)**. Domen sistema za upravljanje zaposlenima modelovan je kroz više iterativnih koraka, pri čemu je svaki naredni korak predstavljao preciznije definisanje poslovnih procesa, entiteta i granica odgovornosti između mikroservisa.

Proces modelovanja obuhvatio je tri faze.

1. Faza 1 – Identifikacija entiteta i veza

U prvoj fazi izvršena je analiza poslovnih procesa sistema za upravljanje ljudskim resursima sa ciljem identifikacije ključnih domen objekata i njihovih međusobnih odnosa.

Identifikovani su osnovni entiteti sistema, uključujući:

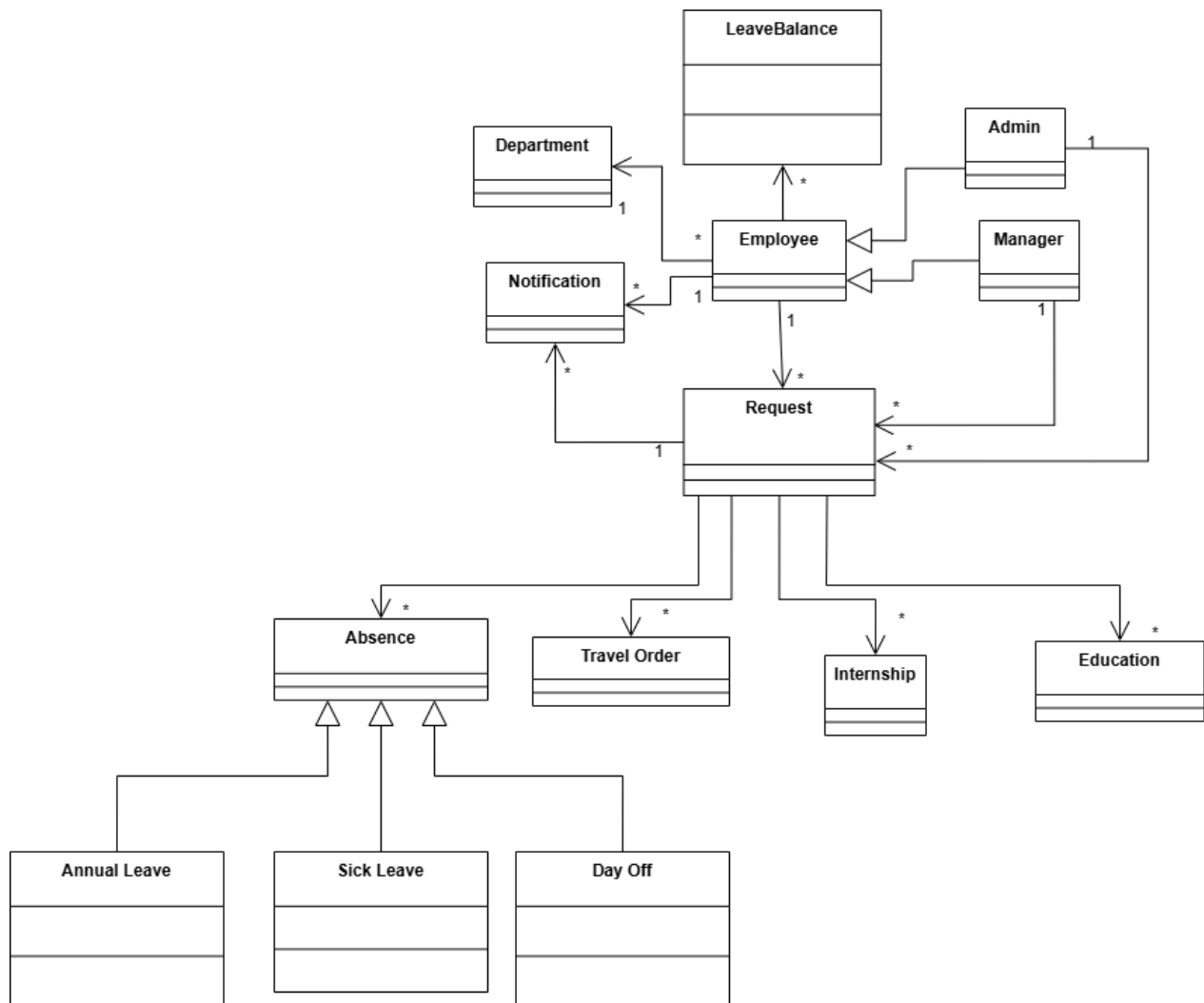
- zaposlenog (Employee)
- odeljenje (Department)
- zahtev (Request)
- odsustvo (Absence)
- godišnji odmor (AnnualLeave)
- bolovanje (SickLeave)
- slobodan dan (DayOff)
- putni nalog (TravelOrder)
- edukaciju (Education)
- praksu (Internship)
- stanje godišnjeg odmora (LeaveBalance)
- notifikaciju (Notification)

Takođe su definisane veze između entiteta, kao što su:

- zaposleni pripada jednom odeljenju
- odeljenje može sadržati više zaposlenih
- zaposleni podnosi zahteve
- zahtev je povezan sa jednim zaposlenim

- zahtev može predstavljati odsustvo, putni nalog, edukaciju ili praksu
- odsustvo pripada određenom zaposlenom
- stanje godišnjeg odmora vezano je za zaposlenog i određenu godinu
- odobren godišnji odmor utiče na stanje raspoloživih dana odmora
- notifikacije se šalju zaposlenima u vezi sa njihovim zahtevima
- korisnički nalog pripada jednom zaposlenom
- menadžer i administrator obrađuju zahteve zaposlenih

Ova faza omogućila je razumevanje strukture domena i poslovnih pravila sistema bez razmatranja implementacionih detalja. Identifikovani entiteti i njihove veze predstavljaju osnovu za dalje modelovanje mikroservisa i definisanje granica odgovornosti unutar sistema.



Slika 10. Model 1: Entiteti i veze

2. Faza 2 – Definisanje agregata

U drugoj fazi primenjen je koncept agregata iz Domain-Driven Design (DDD) pristupa, sa ciljem identifikacije granica konzistentnosti i odgovornosti unutar sistema za upravljanje ljudskim resursima.

Agregat predstavlja grupu međusobno povezanih objekata koji se posmatraju kao jedinstvena celina, pri čemu jedan objekat ima ulogu agregatnog korena (Aggregate Root). Sve izmene i pristup podacima unutar agregata realizuju se preko agregatnog korena, čime se obezbeđuje konzistentnost poslovnih pravila.

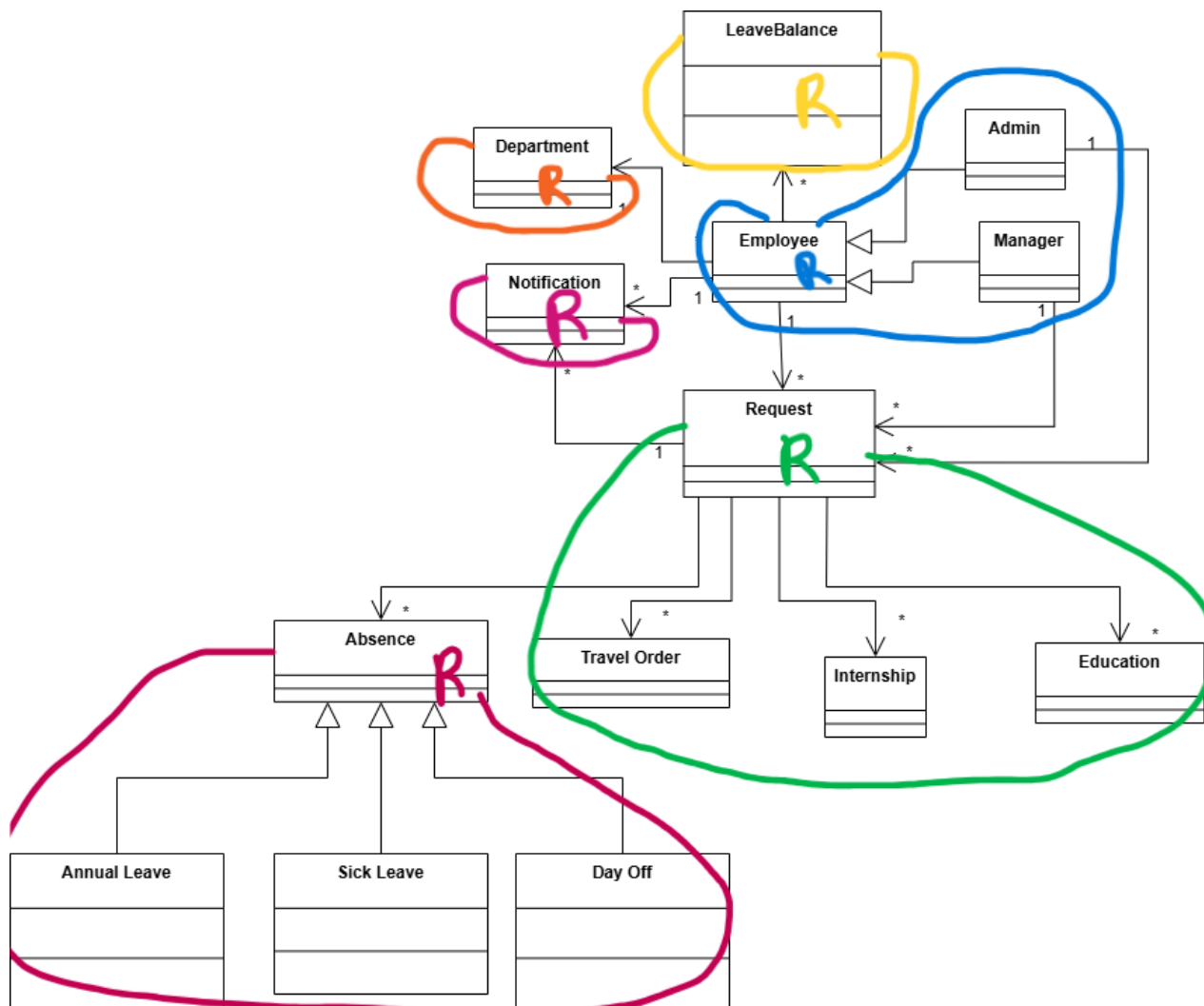
U modelu sistema definisani su sledeći ključni agregati:

- **Employee** – predstavlja agregatni koren za podatke o zaposlenom, korisničkom nalogu i pripadnosti odeljenju
- **Department** – upravlja organizacionom strukturom i zaposlenima koji pripadaju određenom odeljenju
- **Request** – obuhvata sve tipove zahteva koje zaposleni mogu podnositi (odsustvo, putni nalog, edukacija, praksa i drugi zahtevi)
- **Absence** – upravlja podacima o odsustvima zaposlenih, uključujući godišnje odmore, bolovanja i slobodne dane
- **LeaveBalance** – upravlja evidencijom raspoloživih, iskorišćenih i prenetih dana godišnjeg odmora
- **Notification** – upravlja notifikacijama i istorijom obaveštenja vezanih za zahteve i druge aktivnosti u sistemu

Ovakva organizacija omogućava:

- jasno definisane granice odgovornosti između poslovnih oblasti
- očuvanje konzistentnosti podataka unutar svakog agregata
- jednostavnije mapiranje agregata na mikroservise
- lakše održavanje i proširivanje sistema
- bolju kontrolu pristupa i izmena podataka
- smanjenje međusobne zavisnosti između poslovnih komponenti

Definisani agregati predstavljaju osnovu za formiranje mikroservisa i omogućavaju da svaki servis bude odgovoran za jednu jasno definisanu poslovnu oblast sistema.



Slika 11. Model 2: Agregati i agregatni koreni <Root>

3. Faza 3 – Finalni prošireni model

U završnoj fazi modelovanja domena sistema za upravljanje ljudskim resursima definisan je kompletan konceptualni model koji, pored entiteta i agregata, uključuje njihove attribute (obeležja), operacije (metode) i Value Object-e. Ovaj model predstavlja most između faze analize poslovnih zahteva i konkretne implementacije mikroservisne arhitekture.

Konačni model predstavlja detaljnu i stabilnu strukturu domena sistema, spremnu za implementaciju kroz definisane mikroservise.

U okviru finalnog modela definisani su ključni domeni sa svojim karakteristikama:

- **Employee:** osnovni podaci o zaposlenom (ime, prezime, uloga, status), operacije za ažuriranje podataka i povezivanje sa odeljenjem i zahtevima
- **Department:** naziv i opis odeljenja, operacije za upravljanje članovima odeljenja

- **Request:** tip zahteva, status, datum kreiranja i obrade, operacije za kreiranje i promenu statusa zahteva
- **Absence:** period odsustva, tip odsustva (godišnji odmor, bolovanje, slobodan dan), validacije preklapanja datuma
- **LeaveBalance:** ukupan broj dana, iskorišćeni i preneti dani, operacije za ažuriranje stanja nakon odobrenja odmora
- **Notification:** sadržaj notifikacije, datum slanja i status pročitano/nepročitano

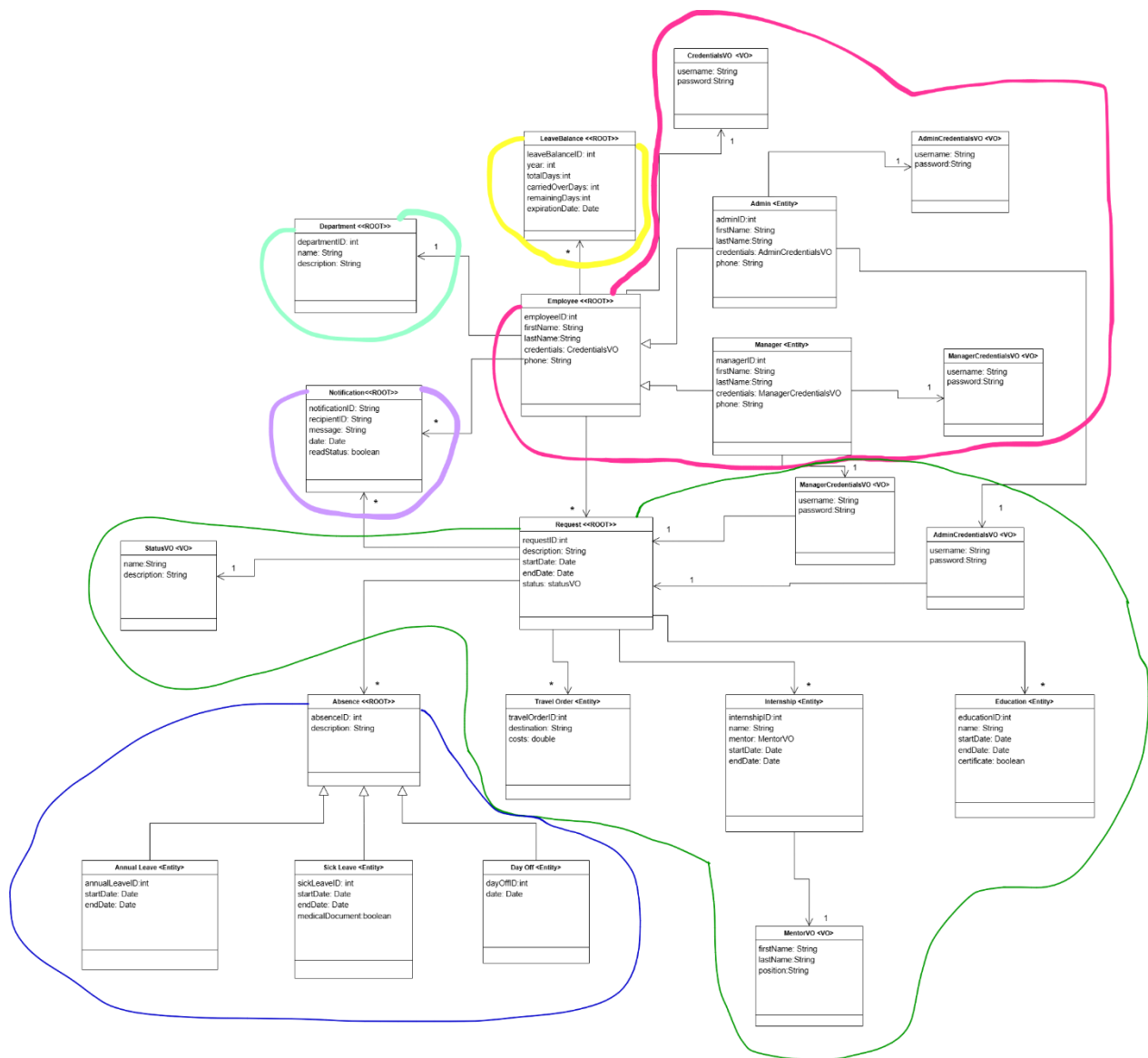
Pored entiteta, definisani su i Value Object-i koji opisuju specifične aspekte domena, kao što su:

- Period (početni i krajnji datum)
- Status zahteva (Pending, Approved, Rejected)
- Tip odsustva (AnnualLeave, SickLeave, DayOff)
- Uloga korisnika (Employee, Manager, Admin)

Ovakav detaljno definisan domen model direktno utiče na strukturu implementacije mikroservisa. Klase, njihovi atributi i operacije predstavljaju osnovu za:

- dizajn baza podataka
- definisanje REST API-ja
- implementaciju poslovne logike u servisima
- mapiranje objekata između mikroservisa

Na ovaj način obezbeđuje se da implementacija verno odražava realne poslovne procese sistema za upravljanje ljudskim resursima, uz jasno definisane granice odgovornosti i konzistentnu arhitekturu.



Slika 12. Model 3: Kompletan model mikroservisne arhitekture

Opis mikroservisa

Sistem za upravljanje ljudskim resursima zasnovan je na mikroservisnoj arhitekturi, pri čemu je celokupna funkcionalnost podeljena na više nezavisnih servisa, od kojih svaki implementira jasno definisanu poslovnu oblast. Ovakav pristup omogućava modularnost sistema, jednostavnije održavanje, kao i nezavisni razvoj i skaliranje pojedinačnih komponenti.

Svaki mikroservis poseduje sopstvenu poslovnu logiku, API interfejs i bazu podataka, čime se obezbeđuje visok stepen autonomije i smanjuje međusobna zavisnost između komponenti sistema.

Komunikacija između mikroservisa realizovana je putem REST arhitekture korišćenjem HTTP protokola i JSON formata za razmenu podataka. Na taj način omogućena je interoperabilnost između servisa, kao i potencijalna integracija sa spoljnim sistemima.

Servisi

Servis za upravljanje zaposlenima (EmployeeService)

EmployeeService predstavlja centralni servis sistema koji upravlja podacima o zaposlenima i korisničkim nalogima. Njegova uloga je da obezbedi evidenciju svih zaposlenih, uključujući njihove osnovne podatke, uloge i status u sistemu.

Ovaj servis omogućava kreiranje novih zaposlenih, izmenu postojećih podataka, kao i njihovo deaktiviranje ili brisanje. Takođe upravlja autentifikacijom i osnovnim korisničkim informacijama.

Podaci koje pruža ovaj servis koriste se u svim ostalim delovima sistema, jer su svi procesi (zahtevi, odsustva i notifikacije) direktno vezani za zaposlenog.

Servis za odeljenja (DepartmentService)

DepartmentService zadužen je za upravljanje organizacionom strukturom sistema. Njegova funkcija je da obezbedi evidenciju odeljenja i povezivanje zaposlenih sa odgovarajućim organizacionim jedinicama.

Ovaj servis omogućava kreiranje i izmenu odeljenja, dodeljivanje zaposlenih odeljenjima i pregled strukture organizacije. Takođe vodi evidenciju o pripadnosti zaposlenih određenim timovima ili sektorima.

Na ovaj način se obezbeđuje jasna hijerarhijska struktura sistema i lakše upravljanje zaposlenima.

Servis za zahteve (RequestService)

RequestService upravlja svim zahtevima koje zaposleni mogu podneti u okviru sistema. Ovaj servis predstavlja centralnu tačku poslovnih procesa odobravanja.

Njegova funkcija obuhvata kreiranje zahteva, praćenje njihovog statusa i upravljanje životnim ciklusom zahteva.

Podržani tipovi zahteva uključuju:

- odsustvo
- godišnji odmor
- bolovanje

- slobodan dan
- putni nalog
- edukaciju
- praksu

Ovaj servis omogućava višestepenu obradu zahteva (npr. menadžersko i administratorsko odobravanje), kao i evidenciju istorije promena statusa i korisnika koji je izvršio obradu.

Servis za odsustva (AbsenceService)

AbsenceService upravlja evidencijom svih vrsta odsustava zaposlenih. Njegova uloga je da obezbedi tačno praćenje perioda odsustva i poštovanje poslovnih pravila sistema.

Ovaj servis omogućava evidenciju godišnjih odmora, bolovanja i slobodnih dana, kao i proveru preklapanja datuma odsustava.

Takođe se koristi za generisanje podataka koji se prikazuju u kalendaru zaposlenog, čime se omogućava jasan pregled svih odsustava i planiranih izostanaka.

Servis za stanje godišnjeg odmora (LeaveBalanceService)

LeaveBalanceService upravlja stanjem godišnjeg odmora zaposlenih. Njegova funkcija je da vodi evidenciju ukupnog broja dana odmora, iskorišćenih dana i prenetih dana iz prethodnih godina.

Ovaj servis omogućava automatsko ažuriranje stanja nakon odobrenja zahteva za godišnji odmor, kao i izračun preostalih dana.

Takođe obezbeđuje pravila za korišćenje prenetih dana, uključujući vremensko ograničenje njihove validnosti.

Servis za notifikacije (NotificationService)

NotificationService zadužen je za obaveštavanje zaposlenih o važnim događajima u sistemu. Njegova funkcija je da obezbedi pravovremenu isporuku informacija korisnicima.

Ovaj servis šalje notifikacije u slučaju promene statusa zahteva, odobrenja ili odbijanja, kao i drugih sistemskih događaja koji zahtevaju pažnju korisnika.

Takođe čuva istoriju poslatih notifikacija, čime se omogućava uvid u sve prethodne aktivnosti i obaveštenja.

Podela sistema na više specijalizovanih mikroservisa omogućava jasno razdvajanje odgovornosti i povećava fleksibilnost razvoja. Svaki servis može se razvijati, testirati i skalirati nezavisno, dok definisana komunikacija putem REST API interfejsa obezbeđuje koherentno

funkcionisanje celokupnog sistema. Ovakva arhitektura predstavlja savremen pristup razvoju informacionih sistema za upravljanje ljudskim resursima i omogućava dugoročnu održivost, proširivost i stabilnost rešenja.

Dokumentovanje API-ja

U okviru projekta izvršeno je detaljno dokumentovanje svih REST API endpoint-a koji omogućavaju komunikaciju sa mikroservisima sistema za upravljanje ljudskim resursima. Dokumentacija obuhvata operacije nad ključnim domen entitetima, pri čemu su za svaku operaciju jasno definisani HTTP metod, ruta (endpoint), parametri zahteva i opis funkcionalnosti. Na ovaj način obezbeđena je transparentnost interfejsa sistema i olakšana integracija frontend aplikacije i ostalih servisa.

API je organizovan po mikroservisima i entitetima, što omogućava preglednu strukturu i dosledan način pristupa podacima. Dokumentacija obuhvata operacije nad entitetima kao što su zaposleni (Employee), odeljenja (Department), zahtevi (Request), odsustva (Absence), stanje godišnjeg odmora (LeaveBalance) i notifikacije (Notification).

Za svaki entitet definisan je skup operacija koje omogućavaju pregled svih zapisa, dohvaćanje pojedinačnog zapisa, kreiranje novih podataka, izmenu postojećih i brisanje zapisa, u skladu sa poslovnim pravilima sistema.

Na primer, za entitet Employee definisane su operacije za dobijanje liste svih zaposlenih, dohvaćanje podataka o konkretnom zaposlenom, kreiranje novog zaposlenog, ažuriranje postojećih informacija i deaktivaciju ili brisanje naloga. Sličan skup operacija postoji i za Department entitet, gde su definisane operacije za upravljanje organizacionim jedinicama i dodeljivanje zaposlenih odeljenjima.

Za Request entitet definisane su operacije za kreiranje zahteva, pregled svih zahteva, filtriranje po zaposlenom i statusu, kao i ažuriranje statusa zahteva u okviru procesa odobravanja. Posebno su definisane rute koje omogućavaju višestepenu obradu zahteva (npr. od strane menadžera i administratora).

Absence entitet obuhvata API operacije za evidenciju odsustava, uključujući godišnje odmore, bolovanja i slobodne dane, kao i proveru validnosti perioda odsustva i sprečavanje preklapanja datuma.

LeaveBalance entitet sadrži operacije za prikaz trenutnog stanja godišnjeg odmora zaposlenog, uključujući ukupan broj dana, iskorišćene i prenete dane, kao i ažuriranje stanja nakon odobrenja zahteva za godišnji odmor.

Notification entitet obuhvata operacije za slanje i pregled notifikacija zaposlenih, kao i označavanje pročitanih obaveštenja i čuvanje istorije notifikacija.

Sve operacije zasnovane su na REST principima, gde svaki resurs ima jedinstvenu URL adresu, a akcije nad resursima definisane su HTTP metodama (GET, POST, PUT, DELETE). Podaci se razmenjuju u JSON formatu, što omogućava jednostavnu i standardizovanu komunikaciju između frontend aplikacije i backend mikroservisa.

Slike u nastavku prikazuju detaljno dokumentovane endpoint-e za pojedinačne entitete, uključujući nazive ruta, HTTP metode i opis funkcionalnosti svake operacije.

Entitet: LeaveBalance			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/LeaveBalance	/	Vraća sve LeaveBalance
GET	api/LeaveBalance /{id}	id	Vraća jedan zapis LeaveBalance na osnovu prosleđenog id-a
POST	api/LeaveBalance	/	Kreira novu LeaveBalance
PUT	api/LeaveBalance /{id}	id	Ažurira LeaveBalance na osnovu prosleđenog id-a
DELETE	api/LeaveBalance /{id}	id	Briše LeaveBalance na osnovu prosleđenog id-a

Slika 13. Dokumentacija API-ja za entitet LeaveBalance

Entitet: Department			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/Department	/	Vraća sve departmane
GET	api/Department /{id}	id	Vraća jedan departman na osnovu prosleđenog id-a
POST	api/Department	/	Kreira novi departman
PUT	api/Department /{id}	id	Ažurira departman na osnovu prosleđenog id-a
DELETE	api/Department /{id}	id	Briše departman na osnovu prosleđenog id-a

Slika 14. Dokumentacija API-ja za entitet Department

Entitet: Notification			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/Notification	/	Vraća sve notifikacije
GET	api/Notification /{id}	id	Vraća jednu notifikaciju na osnovu prosleđenog id-a
POST	api/Notification	/	Kreira novu notifikaciju
PUT	api/Notification /{id}	id	Ažurira notifikaciju na osnovu prosleđenog id-a
DELETE	api/Notification /{id}	id	Briše notifikaciju na osnovu prosleđenog id-a

Slika 15. Dokumentacija API-ja za entitet Notification

Entitet: Employee			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/Employee	/	Vraća sve zaposlene
GET	api/Employee /{id}	id	Vraća jednog zaposlenog na osnovu prosleđenog id-a
POST	api/Employee	/	Kreira novog zaposlenog
PUT	api/Employee /{id}	id	Ažurira zaposlenog na osnovu prosleđenog id-a
DELETE	api/Employee /{id}	id	Briše zaposlenog na osnovu prosleđenog id-a

Slika 16. Dokumentacija API-ja za entitet Employee

Entitet: Admin			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/Admin	/	Vraća sve admina
GET	api/Admin /{id}	id	Vraća jednog admina na osnovu prosleđenog id-a
POST	api/Admin	/	Kreira novog admina
PUT	api/Admin /{id}	id	Ažurira admina na osnovu prosleđenog id-a
DELETE	api/Admin /{id}	id	Briše admina na osnovu prosleđenog id-a

Slika 17. Dokumentacija API-ja za entitet Admin

Entitet: Manager			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/Manager	/	Vraća sve menadžere
GET	api/Manager /{id}	id	Vraća jednog menadžera na osnovu prosleđenog id-a
POST	api/Manager	/	Kreira novog menadžera
PUT	api/Manager /{id}	id	Ažurira menadžera na osnovu prosleđenog id-a
DELETE	api/Manager /{id}	id	Briše menadžera na osnovu prosleđenog id-a

Slika 18. Dokumentacija API-ja za entitet Manager

Entitet: Request			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/Request	/	Vraća sve zahteve
GET	api/Request /{id}	id	Vraća jedan zahtev na osnovu prosleđenog id-a
POST	api/Request	/	Kreira novi zahtev
PUT	api/Request /{id}	id	Ažurira zahtev na osnovu prosleđenog id-a
DELETE	api/Request /{id}	id	Briše zahtev na osnovu prosleđenog id-a

Slika 19. Dokumentacija API-ja za entitet Request

Entitet:	TravelOrder		
HTTP Metoda	Ruta	Parametri	Opis
GET	api/TravelOrder	/	Vraća sve putne naloge
GET	api/TravelOrder /{id}	id	Vraća jedan putni nalog na osnovu prosleđenog id-a
POST	api/TravelOrder	/	Kreira novi putni nalog
PUT	api/TravelOrder /{id}	id	Ažurira putni nalog na osnovu prosleđenog id-a
DELETE	api/TravelOrder /{id}	id	Briše putni nalog na osnovu prosleđenog id-a

Slika 20. Dokumentacija API-ja za entitet TravelOrder

Entitet:	Internship		
HTTP Metoda	Ruta	Parametri	Opis
GET	api/Internship	/	Vraća sve prakse
GET	api/Internship /{id}	id	Vraća jednu praksu na osnovu prosleđenog id-a
POST	api/Internship	/	Kreira novu praksu
PUT	api/Internship /{id}	id	Ažurira praksu na osnovu prosleđenog id-a
DELETE	api/Internship /{id}	id	Briše praksu na osnovu prosleđenog id-a

Slika 21. Dokumentacija API-ja za entitet Internship

Entitet: Education			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/Education	/	Vraća sve edukacije
GET	api/Education /{id}	id	Vraća jednu edukaciju na osnovu prosleđenog id-a
POST	api/Education	/	Kreira novu edukaciju
PUT	api/Education /{id}	id	Ažurira edukaciju na osnovu prosleđenog id-a
DELETE	api/Education /{id}	id	Briše edukaciju na osnovu prosleđenog id-a

Slika 22. Dokumentacija API-ja za entitet Education

Entitet: Absence			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/Absence	/	Vraća sva odsustva
GET	api/Absence /{id}	id	Vraća jedno odsustvo na osnovu prosleđenog id-a
POST	api/Absence	/	Kreira novo odsustvo
PUT	api/Absence /{id}	id	Ažurira odsustvo na osnovu prosleđenog id-a
DELETE	api/Absence /{id}	id	Briše odsustvo na osnovu prosleđenog id-a

Slika 23. Dokumentacija API-ja za entitet Absence

Entitet: AnnualLeave			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/AnnualLeave	/	Vraća sve godišnje odmore
GET	api/AnnualLeave /{id}	id	Vraća jedan godišnji odmor na osnovu prosleđenog id-a
POST	api/AnnualLeave	/	Kreira novi godišnji odmor
PUT	api/AnnualLeave /{id}	id	Ažurira godišnji odmor na osnovu prosleđenog id-a
DELETE	api/AnnualLeave /{id}	id	Briše godišnji odmor na osnovu prosleđenog id-a

Slika 24. Dokumentacija API-ja za entitet AnnualLeave

Entitet: SickLeave			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/SickLeave	/	Vraća sva bolovanja
GET	api/SickLeave /{id}	id	Vraća jedno bolovanje na osnovu prosleđenog id-a
POST	api/SickLeave	/	Kreira novo bolovanje
PUT	api/SickLeave /{id}	id	Ažurira bolovanje na osnovu prosleđenog id-a
DELETE	api/SickLeave /{id}	id	Briše bolovanje na osnovu prosleđenog id-a

Slika 25. Dokumentacija API-ja za entitet SickLeave

Entitet: DayOff			
HTTP Metoda	Ruta	Parametri	Opis
GET	api/DayOff	/	Vraća sve slobodne dane
GET	api/DayOff /{id}	id	Vraća jedan slobodan dan na osnovu prosleđenog id-a
POST	api/DayOff	/	Kreira novi slobodan dan
PUT	api/DayOff /{id}	id	Ažurira slobodan dan na osnovu prosleđenog id-a
DELETE	api/DayOff /{id}	id	Briše slobodan dan na osnovu prosleđenog id-a

Slika 26. Dokumentacija API-ja za entitet DayOff

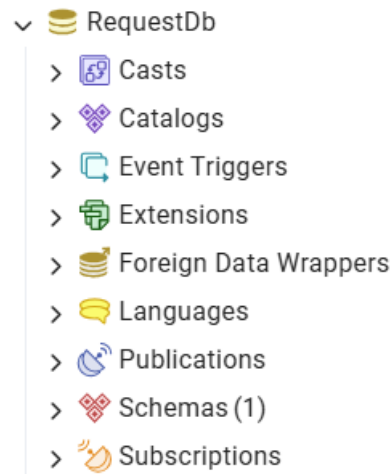
Opis baze podataka

Sistem za upravljanje zaposlenima i odsustvima koristi relacione baze podataka za trajno skladištenje podataka. Kao sistem za upravljanje bazama podataka odabran je **PostgreSQL**, koji obezbeđuje visoku pouzdanost, stabilnost i podršku za rad sa složenim transakcijama. Ovaj sistem omogućava efikasno čuvanje velikih količina strukturiranih podataka i obezbeđuje integritet informacija kroz mehanizme ograničenja i referencijalnih veza.

U skladu sa mikroservisnom arhitekturom, svaki mikroservis poseduje sopstvenu bazu podataka, čime se obezbeđuje potpuna nezavisnost servisa i sprečava direktna zavisnost između različitih delova sistema. Ovakav pristup omogućava autonomiju u razvoju, lakše održavanje i mogućnost skaliranja pojedinačnih komponenti bez uticaja na ostatak sistema.

Struktura podataka

Podaci u bazi organizovani su u relacione tabele koje predstavljaju entitete domen modela. Svaka tabela sadrži primarni ključ koji jednoznačno identifikuje zapis, kao i skup atributa koji opisuju karakteristike tog entiteta. Veze između entiteta modelovane su putem stranih ključeva, čime se obezbeđuje referencijalni integritet podataka.



Slika 27. Baza RequestDb u pgAdmin

Primeri tipičnih entiteta čiji se podaci čuvaju u bazama mikroservisa uključuju:

- **Zaposleni i njihovi nalozi** (EmployeeDb): profilne informacije zaposlenih (ime, prezime, email, telefon, uloga) i kredencijali za autentifikaciju.
- **Departmani / Sektori** (DepartmentDb): nazivi odeljenja, opisi i menadžeri zaduženi za pojedinačne sektore.
- **Zahtevi za odsustva i putne naloge** (RequestDb): tipovi zahteva (odsustvo, službeno putovanje, praksa, edukacija) i prateći detalji (odredišta, troškovi, mentori).
- **Evidencija odobrenih odsustava** (AbsenceDb): godišnji odmori, slobodni dani i bolovanja sa priloženom medicinskom dokumentacijom.
- **Stanje godišnjih odmora** (LeaveBalanceDb): ukupan broj dana, preneti dani iz prethodne godine i datumi njihovog isteka po godinama za svakog zaposlenog.
- **Sistemske notifikacije** (NotificationDb): poruke poslate zaposlenima ili administratorima, sa oznakom o statusu čitanja.

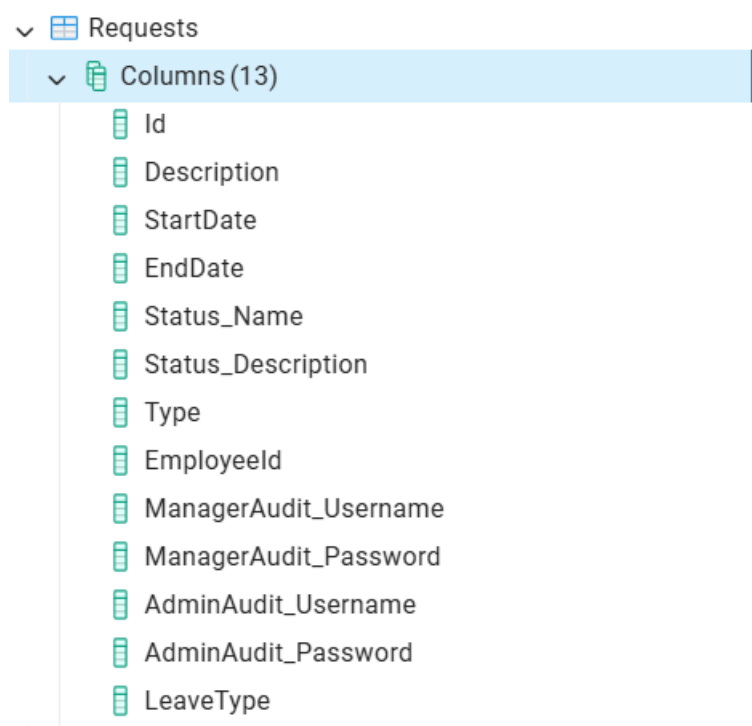
Ovakva struktura omogućava potpunu reprezentaciju poslovnih procesa ljudskih resursa (HR) i evidencije odsustava u digitalnom obliku.

Integritet i konzistentnost podataka

Baza podataka koristi mehanizme integriteta kako bi se obezbedila tačnost i pouzdanost informacija. Ograničenja na nivou baze sprečavaju unos nevalidnih podataka i održavaju konzistentnost sistema.

Korišćeni mehanizmi uključuju:

- **Primarne ključeve** za jedinstvenu identifikaciju zapisa (npr. Absenceld u tabeli Absences, RequestId u tabeli Requests).
- **Strane ključeve** za očuvanje referencijalnog integriteta (npr. mapiranje EmployeeId na zapise iz servisa zaposlenih).
- **Ograničenja nad vrednostima atributa** (npr. proveravanje opsega datuma gde početak odsustva mora biti pre kraja, numerička ograničenja nad poljima za telefone i budžete).
- **Transakcije** koje obezbeđuju atomarnost operacija i ACID svojstva.



Slika 28. Prikaz kolona baze RequestDb

Na taj način se sprečava nastanak nedoslednosti, posebno u situacijama kada više korisnika istovremeno pristupa sistemu ili kada se kreiraju zahtevi sa kritičnim vremenskim preklapanjima.

Izolacija podataka po servisima

Svaki mikroservis upravlja sopstvenim podacima i odgovoran je za njihovu validnost i konzistentnost. Servisi ne pristupaju direktno bazama drugih servisa, već razmenjuju informacije isključivo putem API interfejsa (HTTP komunikacija i API Gateway). Time se obezbeđuje slaba sprega (loose coupling) između komponenti i smanjuje rizik od neželjenih efekata prilikom promena u strukturi podataka.

Ovakav model omogućava:

- Nezavisnu evoluciju šeme baze svakog servisa (migracije se primenjuju pojedinačno).
- Bolju bezbednost podataka jer su baze izolovane unutar interne Docker mreže.
- Jednostavnije testiranje i razvoj nezavisnih komponenti.
- Lakšu migraciju i horizontalno skaliranje kritičnih baza podataka.

Pristup bazi podataka

Backend servisi komuniciraju sa bazom podataka putem odgovarajućih biblioteka za pristup podacima, koje omogućavaju izvršavanje upita, manipulaciju zapisima i upravljanje transakcijama. Ove biblioteke pojednostavljaju rad sa bazom i obezbeđuju apstrakciju nad SQL upitima.

Pristup bazi podataka realizovan je korišćenjem **Entity Framework Core** biblioteke, koja omogućava rad sa bazom kroz objektno-relaciono mapiranje (ORM).

Bezbednost podataka

Bezbednost baza podataka obezbeđena je kontrolom pristupa i ograničavanjem dozvola. Samo autorizovani kontejneri servisa unutar Docker Compose mreže (microservices-net) mogu uspostaviti konekciju sa odgovarajućom bazom podataka, dok spoljni svet i krajnji korisnici nemaju direktan pristup portovima baza (5432). Osetljive informacije i lozinke se prosleđuju kao bezbedne varijable okruženja (environment variables) tokom pokretanja kontejnera.

Izabrana struktura baze podataka omogućava pouzdano, konzistentno i skalabilno skladištenje informacija neophodnih za rad sistema. Kombinacija relacione baze PostgreSQL i mikroservisnog pristupa obezbeđuje stabilnu osnovu za dalji razvoj sistema, uz mogućnost lakog prilagođavanja budućim poslovnim i revizionim zahtevima.

Opis komunikacije između servisa

U sistemu za upravljanje ljudskim resursima komunikacija između komponenti realizovana je u skladu sa principima mikroservisne arhitekture. Svaki mikroservis predstavlja autonomnu celinu koja implementira sopstvenu poslovnu logiku i upravlja sopstvenim podacima, dok se međusobna saradnja ostvaruje putem jasno definisanih API interfejsa.

Komunikacija između servisa zasnovana je na REST arhitekturi i ostvaruje se putem HTTP/HTTPS protokola, uz korišćenje JSON formata za razmenu podataka. Ovakav pristup omogućava interoperabilnost, jednostavnu integraciju i nezavisnost servisa, jer nijedan servis ne pristupa direktno bazi podataka drugog servisa. Umesto toga, svi zahtevi za podacima ili poslovnim operacijama prosleđuju se putem API poziva.

Servisi međusobno komuniciraju sinhrono, što znači da servis koji šalje zahtev čeka odgovor drugog servisa pre nastavka izvršavanja operacije. Ovaj model je pogodan za operacije koje zahtevaju trenutnu validaciju ili povratnu informaciju, kao što su kreiranje zahteva, provera zaposlenog ili ažuriranje stanja godišnjeg odmora.

Na primer, prilikom kreiranja zahteva u **RequestService**, servis komunicira sa **EmployeeService** kako bi proverio validnost zaposlenog, kao i sa **AbsenceService** ili **LeaveBalanceService** u zavisnosti od tipa zahteva. Nakon obrade zahteva, **NotificationService** se poziva kako bi se zaposlenom poslalo obaveštenje o promeni statusa.

Komunikacija između frontend aplikacije i backend mikroservisa takođe se odvija putem REST API-ja. Korisnički interfejs šalje zahteve odgovarajućem servisu, koji obrađuje zahtev, pristupa sopstvenoj bazi podataka i vraća rezultat u strukturisanom JSON formatu. Na ovaj način obezbeđena je jasna podela odgovornosti između prezentacionog sloja i poslovne logike sistema.

Bezbednost komunikacije obezbeđena je korišćenjem HTTPS protokola, koji omogućava šifrovanje prenosa podataka između servisa i sprečava neovlašćen pristup ili presretanje informacija. Pored toga, sistem koristi mehanizme autentifikacije i autorizacije zasnovane na ulogama (Employee, Manager, Admin), čime se osigurava da samo ovlašćeni korisnici i servisi mogu pristupiti određenim funkcionalnostima.

Komunikacija između servisa zasnovana na REST principima omogućava fleksibilno i pouzdano funkcionisanje sistema za upravljanje ljudskim resursima. Jasno definisani API interfejsi, nezavisnost podataka i upotreba standardnih protokola doprinose skalabilnosti, održivosti i otpornosti sistema na greške, čime se obezbeđuje stabilan rad u realnim poslovnim uslovima.

Opis frontend-backend interakcije

Frontend i backend komponenta sistema za upravljanje ljudskim resursima povezane su kroz jasno definisan model komunikacije zasnovan na REST arhitekturi. Korisnički interfejs razvijen u React biblioteci predstavlja prezentacioni sloj sistema i zadužen je za interakciju sa korisnikom, dok backend mikroservisi implementirani u okviru .NET platforme obezbeđuju poslovnu logiku i pristup podacima.

Komunikacija između frontend aplikacije i backend servisa ostvaruje se putem HTTP/HTTPS protokola, pri čemu frontend šalje zahteve odgovarajućim API endpoint-ima, a backend vraća strukturisane odgovore u JSON formatu. Ovakav pristup omogućava jasnu separaciju

odgovornosti između slojeva sistema i nezavisnost razvoja korisničkog interfejsa i serverske logike.

Kada korisnik izvrši određenu akciju u aplikaciji, kao što je pregled zaposlenih, podnošenje zahteva, pregled odsustava ili praćenje stanja godišnjeg odmora, frontend komponenta generiše HTTP zahtev koji se prosleđuje odgovarajućem mikroservisu (EmployeeService, RequestService, AbsenceService, LeaveBalanceService ili NotificationService). Backend servis obrađuje zahtev, izvršava potrebne operacije nad bazom podataka i vraća rezultat koji frontend interpretira i prikazuje korisniku. Na taj način obezbeđuje se dinamičko ažuriranje sadržaja bez potrebe za ponovnim učitavanjem cele stranice.

Frontend aplikacija funkcioniše kao Single Page Application (SPA), što znači da se većina navigacije i prikaza sadržaja odvija na klijentskoj strani. Backend servisi pružaju isključivo podatke i poslovne operacije, dok je logika prikaza implementirana u React komponentama. Ovakav model doprinosi boljem korisničkom iskustvu, bržem odzivu sistema i smanjenju opterećenja servera.

Razmena podataka između frontend-a i backend-a obuhvata različite tipove operacija, uključujući dohvaćanje podataka, kreiranje novih zapisa, izmenu postojećih informacija i brisanje podataka. Backend servisi koriste standardne HTTP metode (GET, POST, PUT, DELETE) kako bi jasno definisali namenu zahteva, dok frontend obrađuje odgovore i prikazuje odgovarajuće poruke korisniku u slučaju uspeha ili greške.

Poseban značaj ima integracija sa NotificationService, koji omogućava da korisnici u realnom vremenu dobijaju obaveštenja o promeni statusa svojih zahteva, odobrenjima ili drugim sistemskim događajima.

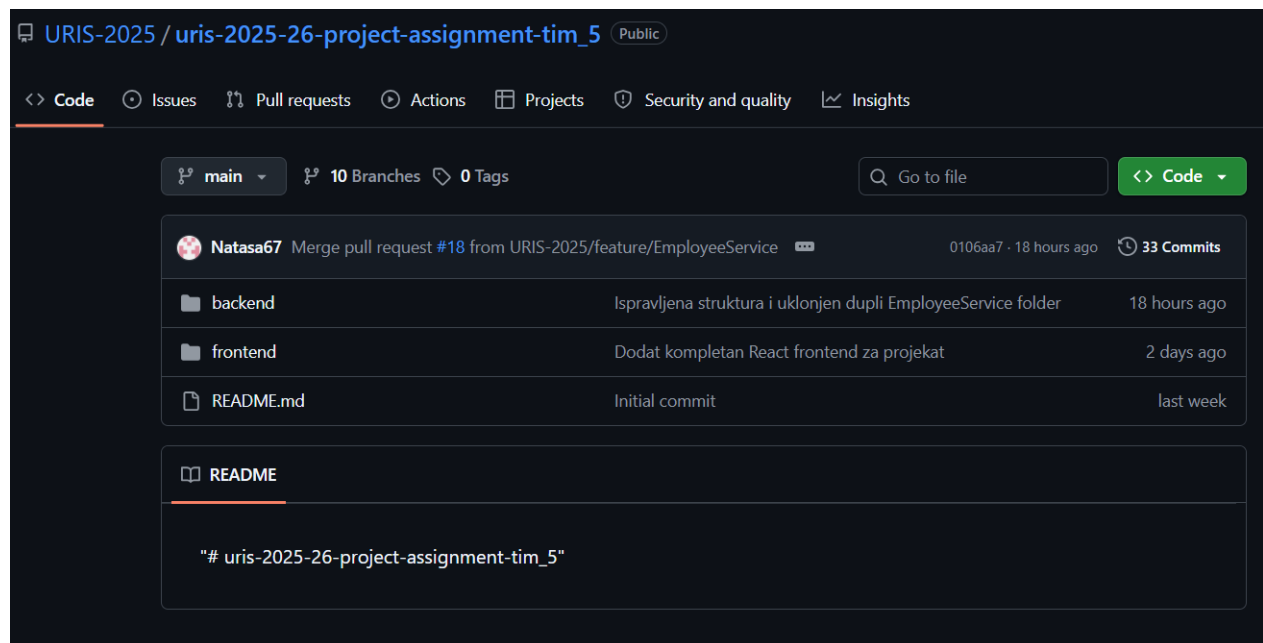
Bezbednost komunikacije obezbeđena je korišćenjem HTTPS protokola, koji omogućava šifrovan prenos podataka između klijenta i servera. Time se sprečava neovlašćen pristup osetljivim informacijama i obezbeđuje integritet prenosa. Takođe, implementirani su mehanizmi autentifikacije i autorizacije zasnovani na ulogama (Employee, Manager, Admin), kako bi se osiguralo da samo ovlašćeni korisnici mogu pristupiti zaštićenim funkcionalnostima sistema.

Frontend-backend komunikacija zasnovana na REST principima i savremenim web tehnologijama omogućava efikasno, sigurno i responzivno funkcionisanje sistema za upravljanje ljudskim resursima. Jasna podela odgovornosti između prezentacionog i serverskog sloja doprinosi skalabilnosti, održivosti i kvalitetu korisničkog iskustva.

Opis DevOps postavke

DevOps postavka sistema za upravljanje ljudskim resursima osmišljena je sa ciljem automatizacije procesa razvoja, testiranja, isporuke i održavanja softvera. Primena DevOps principa omogućava kontinuiranu integraciju i isporuku sistema, čime se obezbeđuje stabilan i pouzdan rad aplikacije, kao i brza implementacija novih funkcionalnosti i ispravki.

Izvorni kod projekta čuva se u distribuiranom sistemu za upravljanje verzijama Git, u okviru udaljenog zajedničkog repozitorijuma na platformi GitHub. Svaki član razvojnog tima radi na sopstvenoj grani (*branch*), dok se integracija promena vrši kroz proces spajanja (*merge*) u glavnu razvojnu granu nakon uspešnog testiranja. Ovakav pristup omogućava paralelan rad više programera na različitim mikroservisima i minimizuje rizik od nastanka konflikata u kodu.



Slika 29. Prikaz GitHub repozitorijuma

Kontinuirana integracija realizovana je korišćenjem GitHub Actions alata, koji omogućava automatsko pokretanje procesa izgradnje projekta prilikom svakog slanja koda (*push*) ili kreiranja zahteva za spajanje (*pull request*) u repozitorijumu. U okviru CI procesa vrši se automatsko kompajliranje svih .NET mikroservisa, provera zavisnosti i validacija osnovne funkcionalnosti sistema. Time se obezbeđuje da kod u glavnoj grani uvek ostane stabilan i spreman za dalju distribuciju i postavljanje na cloud infrastrukturu.

Za pokretanje, testiranje i orkestraciju sistema u razvojnim i produkcionim uslovima koristi se kontejnerizacija pomoću Docker tehnologije. Kompletan sistem je modularno podeljen i definisan kroz centralni docker-compose.yml fajl koji obuhvata ukupno 14 kontejnera: 6 .NET mikroservisa, 6 izolovanih PostgreSQL baza podataka, Ocelot API Gateway i Nginx frontend

poslužitelj. Svaki mikroservis pokreće se u zasebnom kontejneru zajedno sa sopstvenom pripadajućom bazom podataka (arhitektura *Database per Service*), čime se obezbeđuje potpuna izolacija komponenti, bezbednost podataka i jednostavno upravljanje zavisnostima.

Opis testiranja

Testiranje sistema za upravljanje zaposlenima i odsustvima sprovedeno je sa ciljem provere ispravnosti funkcionalnosti, stabilnosti rada i pouzdanosti mikroservisne arhitekture.

Proces testiranja obuhvatao je više nivoa - od pojedinačnih komponenti i domenskih pravila do međusobne komunikacije servisa i interakcije sa korisničkim interfejsom.

Na nivou backend mikroservisa sprovedeno je automatsko testiranje korišćenjem xUnit okvira, koji predstavlja industrijski standard za .NET ekosistem. Za svaki mikroservis razvijen je poseban skup jediničnih testova (unit testova), kojima su proverene ključne funkcionalnosti poslovne logike, validacija podataka i pravilno rukovanje izuzecima. Ovakav pristup omogućava izolovano testiranje komponenti i brzo otkrivanje regresionih grešaka tokom razvoja.

```
namespace Absence.Domain.Tests;

using System;
using Absence.Domain.Entities;
using Xunit;

public class AnnualLeaveTests
{
    [Fact]
    public void Create_ValidDates_ShouldCreateInstance()
    {
        // Arrange
        var startDate = DateTime.Now;
        var endDate = startDate.AddDays(2);

        // Act
        var leave = new AnnualLeave(1, "Vacation", startDate, endDate);

        // Assert
        Assert.NotNull(leave);
        Assert.Equal("Vacation", leave.Description);
        Assert.Equal(startDate, leave.StartDate);
        Assert.Equal(endDate, leave.EndDate);
    }
}
```

Slika 30. Klasa *AnnualLeaveTests*, kreiranje entiteta i validacije

```
[Fact]
public void Create_StartDateAfterEndDate_ShouldThrowArgumentException()
{
    // Arrange
    var startDate = DateTime.Now.AddDays(2);
    var endDate = DateTime.Now;

    // Act & Assert
    Assert.Throws<ArgumentException>(() =>
        new AnnualLeave(1, "Invalid Vacation", startDate, endDate));
}
```

Slika 31. Prikaz pojedinih xUnit testova klase AnnualLeaveTests

```
public class SickLeaveTests
{
    [Fact]
    public void Create_SickLeave_StoresMedicalDocument()
    {
        // Arrange & Act
        var leave = new SickLeave(1, "Flu", DateTime.Now, DateTime.Now.AddDays(1),
true);

        // Assert
        Assert.True(leave.MedicalDocument);
    }
}
```

Slika 32. Prikaz pojedinih testova unutar Absence.Domain.Tests projekta

Pored jediničnih testova, za mikroservise su realizovani i integracioni testovi, kojima je proverena ispravnost rada sistema u uslovima stvarne interakcije između komponenti. Integracioni testovi obuhvatali su simulaciju HTTP request/response ciklusa kontrolera, rad sa skladištem podataka (repozitorijumima) i proveru celokupnog toka obrade zahteva, uključujući upload i download prateće dokumentacije za bolovanja. Na ovaj način potvrđeno je da mikroservisi funkcionišu ispravno ne samo pojedinačno, već i u okviru šireg sistema.

```

[Fact]
public async Task UploadDocument_ValidFile_SavesFileAndUpdatesAbsence()
{
    // 1. Arrange: Kreiranje bolovanja (SickLeave) u bazi koje inicijalno nema
    dokument
    var sickLeave = new SickLeave(1, "Sick with flu", DateTime.Now.AddDays(-1),
    DateTime.Now.AddDays(2), false);
    await _repository.AddAsync(sickLeave);
    int absenceId = sickLeave.AbsenceId;

    // Priprema simuliranog (Fake) fajla za upload
    var content = "This is a test medical document content.";
    var bytes = Encoding.UTF8.GetBytes(content);
    var fileStream = new MemoryStream(bytes);
    var file = new FormFile(fileStream, 0, bytes.Length, "file", "test_report.pdf")
    {
        Headers = new HeaderDictionary(),
        ContentType = "application/pdf"
    };

    // 2. Act: Poziv akcije kontrolera za upload dokumenta
    var result = await _controller.UploadDocument(absenceId, file);

    // 3. Assert: Provera da li je statusni kod 200 OK i da li je fajl sačuvan na
    disku
    var okResult = Assert.IsType<OkObjectResult>(result);
    Assert.NotNull(okResult.Value);

    // Provera da li fajl fizički postoji u 'uploads' folderu

    var expectedFileNamePrefix = $"absence_{absenceId}_";
    var files = Directory.GetFiles(_uploadsFolder, $"{expectedFileNamePrefix}*");
    Assert.Single(files);
    Assert.EndsWith("test_report.pdf", files[0]);

    // Provera da li je MedicalDocument status u bazi promenjen na 'true'
    var updatedAbsence = (SickLeave)await _repository.GetByIdAsync(absenceId);
    Assert.True(updatedAbsence.MedicalDocument);
}

```

Slika 33. Prikaz integracionog testa klase *AbsencesControllerTests* za upload fajlova

Funkcionalnost REST API interfejsa dodatno je proveravana korišćenjem ugrađenog alata Swagger, koji je dostupan na svakom pojedinačnom mikroservisu i API Gateway-u u razvojnom okruženju. Swagger omogućava vizuelni pregled ruta, slanje testnih HTTP zahteva

i validaciju strukture odgovora. Testiranjem su obuhvaćeni različiti scenariji, uključujući uspešne GET/POST/PUT/DELETE operacije, greške pri neispravnim datumima (preklapanje odsustva), zahteve za nepostojeće zaposlene i nepravilne ulazne podatke. Posebna pažnja posvećena je proveru HTTP statusnih kodova (200 OK, 201 Created, 400 BadRequest, 404 NotFound) i strukture vraćenih JSON odgovora.

Komunikacija između mikroservisa testirana je kroz scenarije koji uključuju sinhronu i asinhronu razmenu podataka. Posebno su provereni sledeći tokovi: Validacija zaposlenih preko HTTP klijenta: Provereno je da li `RequestService` uspešno komunicira sa `EmployeeService` kako bi verifikovao postojanje zaposlenog pre nego što mu odobri kreiranje zahteva za putni nalog ili odsustvo.

Sistemske notifikacije: Testirana je integracija sa `NotificationService` kako bi se osiguralo da se notifikacije (npr. kada menadžer odobri zahtev radnika ili kada menadžer podnese sopstveni zahtev) pravilno kreiraju, šalju primaocu i čuvaju u bazi podataka bez ometanja glavnog toka transakcije.

Testiranje baze podataka obuhvatilo je proveru integriteta podataka, validnost referencijalnih veza i pravilno izvršavanje Entity Framework Core migracija u Docker okruženju. Provereno je da sve operacije nad podacima ostavljaju bazu u konzistentnom stanju i da konkurentni pristupi (npr. istovremeno kreiranje zahteva od strane više radnika) ne narušavaju definisana poslovna pravila.

Frontend deo sistema (React aplikacija) testiran je kroz verifikaciju korisničkog interfejsa i ispravnosti komunikacije sa backend servisima preko API Gateway-a. Posebna pažnja posvećena je:

Pravilnom prikazu i filtriranju podataka: Provereno je da li zaposleni vide samo svoja odsustva, menadžeri odsustva svog sektora, a administratori celokupan registar. Validaciji formi: Testirano je reagovanje aplikacije na prazna polja, unos ne-numeričkih karaktera u polje za telefon (koji se automatski filtriraju u realnom vremenu), i onemogućavanje ("disable") dugmadi za slanje dok se ne ispune svi obavezni uslovi i prilože dokumenti (kod bolovanja).

Tokom testiranja korišćeni su realistični testni podaci koji simuliraju tipične scenarije rada u preduzeću, poput evidentiranja godišnjeg odmora, uvoza medicinske dokumentacije za bolovanje, odobravanja službenih putovanja i praćenja preostalog broja dana odmora po godinama. Time je obezbeđena verodostojna procena ponašanja sistema u realnim uslovima rada.

Rezultati sprovedenih testova pokazali su da sistem ispravno implementira sve definisane

funkcionalnosti i da je sposoban za stabilan rad u uslovima očekivanog opterećenja. Identifikovane greške i neusklađenosti su otklonjene tokom razvoja, čime je unapređena ukupna pouzdanost i kvalitet aplikacije.

Proces testiranja predstavlja ključan deo razvoja softvera, jer omogućava rano otkrivanje problema i smanjenje rizika od grešaka u produkcionom okruženju. Kontinuirana verifikacija funkcionalnosti doprinosi dugoročnoj održivosti sistema i spremnosti za dalju evoluciju i proširenja poslovne logike.

Zaključak

Razvijeni sistem za upravljanje ljudskim resursima predstavlja savremeno softversko rešenje zasnovano na principima mikroservisne arhitekture. Sistem omogućava digitalizaciju ključnih poslovnih procesa, uključujući upravljanje zaposlenima, odeljenjima, zahtevima, odsustvima, stanjem godišnjeg odmora i notifikacijama. Implementacijom centralizovanog sistema obaveštavanja i evidencije obezbeđeno je praćenje aktivnosti korisnika i lakše upravljanje poslovnim procesima.

Primena mikroservisnog pristupa omogućila je razdvajanje sistema na autonomne komponente, čime su postignuti fleksibilnost, skalabilnost i jednostavnije održavanje. Svaki mikroservis upravlja sopstvenim podacima i poslovnom logikom, dok se međusobna komunikacija ostvaruje putem jasno definisanih REST API interfejsa. Ovakva arhitektura omogućava nezavisnu evoluciju pojedinačnih delova sistema bez narušavanja funkcionalnosti celokupne aplikacije.

Backend deo sistema razvijen je korišćenjem .NET platforme i C# programskog jezika, dok je za skladištenje podataka korišćena relacijska baza podataka. Frontend aplikacija implementirana je kao Single Page Application zasnovana na React biblioteci, čime je obezbeđen moderan i responzivan korisnički interfejs. Kombinacija ovih tehnologija omogućava visoke performanse, stabilnost i jednostavnu integraciju sa drugim sistemima.

Posebna pažnja posvećena je kvalitetu softvera kroz pravilno modelovanje domena korišćenjem DDD pristupa, kao i kroz definisanje jasnih poslovnih pravila u okviru svakog mikroservisa. Na taj način obezbeđena je konzistentnost podataka i pouzdanost poslovne logike sistema.

Dodatno, funkcionalnost API-ja i komunikacije između servisa proveravana je kroz praktične scenarije korišćenja, što je doprinelo stabilnosti sistema u realnim uslovima i pravilnom funkcionisanju svih poslovnih procesa.

Sistem takođe podržava višeslojnu kontrolu pristupa zasnovanu na ulogama (Employee, Manager, Admin), čime je obezbeđena sigurnost i jasno definisane granice odgovornosti korisnika u okviru sistema.

Razvijeni sistem predstavlja kvalitetnu osnovu za dalji razvoj i proširenje funkcionalnosti. U budućnosti je moguće unaprediti bezbednosne mehanizme, uvesti naprednije analitičke module, poboljšati automatizaciju procesa odobravanja zahteva, kao i integraciju sa spoljnim servisima i mobilnim aplikacijama. Takođe, arhitektura sistema omogućava dodavanje novih mikroservisa bez značajnih promena u postojećem rešenju.

Na osnovu sprovedene analize, modelovanja i definisanja arhitekture može se zaključiti da sistem uspešno ispunjava postavljene ciljeve i predstavlja stabilno, skalabilno i moderno softversko rešenje za upravljanje ljudskim resursima.